



CEN207 Term Project

Project Guide — Fall 2026-2027

Asst. Prof. Dr. Uğur CORUH

One project, twice

- **One project** this term: an application you choose from the list
- Implemented **twice** with the data structures and algorithms from class
- First in **C** (midterm check), then extended in **Java** (final check)
- Goal: answer "why did I choose this structure?" with **numbers** — complexity and measurements, not opinions

At a glance — team and checks

- **Team:** at most 5 students (working alone is also allowed)
- Each topic taken by **one team only**; no team changes after week 3
- **Midterm check (RAP1):** C implementation, report, demo — week 7, 30.10.2026 — 60% of midterm
- **Final check (RAP2):** Java implementation, report, demo — week 15, 25.12.2026 — 70% of final

At a glance — tools, scope, deadlines

- **Tools:** CMake + GoogleTest + Doxygen (C); JDK 21 + Maven + JUnit 5 (Java); Git and GitHub
- **Scope:** every requirement code (V1–V6, F1–F9) is mandatory for every topic
- The topic box tells you what each code represents in that application
- **Deadlines:** topic/team selection by end of week 3 (04.10.2026); plan approval week 4 (09.10.2026)



1. Calendar

Project calendar

Milestone	Week	Date
Topic and team selection	end of week 3	04.10.2026
Project plan approval	week 4	09.10.2026
Midterm demo + interim report (RAP1)	week 7	30.10.2026
Quiz-1	week 8	31.10–08.11.2026
Final demo + final report (RAP2)	week 15	25.12.2026
Quiz-2	week 16	04–17.01.2027



2. Assessment Structure

How the project feeds into the grade

$$Grade_{Midterm} = 0.6 \cdot RAP1 + 0.4 \cdot Quiz-1$$

$$Grade_{Final} = 0.7 \cdot RAP2 + 0.3 \cdot Quiz-2$$

$$Course\ grade = 0.4 \cdot Midterm + 0.6 \cdot Final$$

Which learning outcomes each checkpoint measures

LO	Bloom level	RAP1	RAP2
LO.1 Linear/non-linear data structures	Understand	✓	✓
LO.2 Time/space complexity (Big-O)	Analyze	✓	✓
LO.3 Sorting and searching	Apply	✓	✓
LO.4 Trees and hash tables	Apply	✓	✓
LO.5 Graph algorithms	Apply	✓	✓
LO.6 File organization	Evaluate	—	✓
LO.7 Structure/algorithm selection	Synthesize	✓	✓



3. Tools and Setup

Tools — C side (midterm)

Tool	What for	Output	Condition
GCC/Clang/MSVC + CMake	Build the C app	Executable (not submitted)	Zero errors, Windows + WSL/Linux
GoogleTest + gcov/lcov	Unit tests, coverage	HTML coverage report	100% statement coverage
Doxygen	Source documentation	PDF	100% coverage; PDF only, no HTML folder

Tools — Java side (final) and shared

Tool	What for	Output	Condition
JDK 21 + Maven	Build the Java app	Release build	<code>mvn clean verify</code> , zero errors
JUnit 5 + JaCoCo	Unit tests, coverage	HTML coverage report	100% statement coverage
Javadoc/Doxygen	Source documentation	PDF	100% coverage; PDF only
Git + GitHub	Version control	Private repository	Meaningful commits, proper <code>.gitignore</code>
GitHub Actions (optional)	Continuous integration	CI status	If enabled, must be green before merging

Create your repository from the template

- **Do not fork:** a fork of a public repository cannot be made private
- Template page: **Use this template** → **Create a new repository**
- Choose the owner, name it with the course code (pattern on the next slide), select **Private**
- Add the **instructor** (ucoruh) and your **teammate** as collaborators

Templates and repository names

Check	Template	Repository name
Midterm (C)	cpp-cmake-ctest-template	cen207-project-name-surname-c
Final (Java)	eclipse-java-maven-template	cen207-project-name-surname-java

Both templates already provide building, unit testing, documentation generation, coverage measurement and packaging — you build **on** them.

Showing your project locally

A private repository on GitHub Free has no GitHub Pages site. You show everything **on your computer**:

- `7-build-all-windows.bat` (Linux/WSL: `7-build-all-linux.sh`) builds, tests and makes every report
- `9-open-site-windows.bat` (Linux: `9-open-site-linux.sh`) opens the full site on `http://localhost` : every report (tests, code coverage, documentation coverage, Windows and Linux) and the API docs
- `release/` holds every output: app/exe, library, reports, API docs, `site.zip` , source, `ASSETS.md` , `SHA256SUMS.txt`
- `10-release-windows.bat` (Linux: `10-release-linux.sh`) makes the GitHub Release; releases work on private repositories

Demo checklist

What you open and show, in this order:

1. The **home page** of the local site (`9-open-site-...` , on `http://localhost`).
2. **Each report page**: tests, code coverage, documentation coverage (Windows and Linux).
3. The **API docs**.
4. The `release/` **folder** listing.
5. **Run the app** from the release archive.
6. The **GitHub Release page** of your private repository (the instructor is a collaborator).

The lib / app / test layout

- **lib** — data structures and algorithms live here
- **app** — console menus and user interaction; uses **lib**
- **test** — unit tests; uses **lib**

Before you start: make sure your environment is ready — see the **Prerequisites** page/deck.



4. Choosing a Topic

Where the topics are

- The **Appendix — Project topic list**, at the end of the project guide page
- **200 topics** in **eight groups** of 25
- Each topic box: a short summary, then one sentence per requirement code saying what it stores and what operation it performs **in that application**

How to choose

1. Browse the list and pick a topic
2. Write your choice in the shared **team and topic list** (link in the Teams post)
— one topic per team, first to write it gets it
3. Get it approved together with your **project plan** — the topic cannot change after approval

Rules about the topic itself

- The mappings in a topic box are a **starting suggestion** — a more natural feature meeting the same requirement is fine, justify it in the report
- An idea **not on the list** is allowed with instructor approval, if it meets every requirement meaningfully
- **Retaking the course:** choose a topic different from your previous project



5. Requirements

Implemented by you, not the library

Each code lists the **week it is taught** and the **LO it measures**. You implement the basic operations of every structure (insert, delete, search, list...) **yourself**.

Ready-made library structures (e.g. Java's `java.util` collections) do **not** count towards these requirements.

5.1 Midterm scope (C, weeks 1–6) — V1–V3

Code	Requirement	Week	LO
V1	Linked list: doubly + XOR/circular; insert, delete, search, traverse both ways	2	LO.1
V2	Sparse matrix: store only filled cells; read, write, walk rows/columns	2	LO.1
V3	Stack and queue: array or linked-list based	3	LO.1

5.1 Midterm scope (C, weeks 1–6) — V4–V6

Code	Requirement	Week	LO
V4	Tree and heap: binary tree + 3 traversals; heap-based priority queue; heap sort	4	LO.1, LO.4
V5	Graph and traversal: adjacency list/matrix; BFS and DFS	5	LO.1, LO.5
V6	Search and hashing: binary search; hash table + collision resolution	6	LO.3, LO.4

5.2 Final scope (Java, weeks 9–14) — F1–F3

Code	Requirement	Week	LO
F1	Port to Java: V1–V6 with generics; all features in one menu	9–14	LO.1, LO.7
F2	Graph algorithms: ≥ 2 of MST/shortest-path/topo-sort/SCC/cycle/max-flow	9	LO.5
F3	Sorting: ≥ 3 algorithms, timed comparison across data sizes	10	LO.2, LO.3

5.2 Final scope (Java, weeks 9–14) — F4–F6

Code	Requirement	Week	LO
F4	BST and AVL: balancing rotations; insert, delete, search, range query	11	LO.4
F5	String algorithms: KMP/Boyer–Moore search + edit distance/LCS	12	LO.1, LO.3
F6	Trie and disjoint sets: prefix tree + union-find	9, 12	LO.4

5.2 Final scope (Java, weeks 9–14) — F7–F9

Code	Requirement	Week	LO
F7	File organization: sequential + direct-access file; collision resolution in the file	13	LO.6
F8	B+ tree index: secondary-key index over file records	14	LO.4, LO.6
F9	Growing files: extendible hashing or external merge sort	14	LO.3, LO.6

5.3 Rules for both checks — the application

- **Console app, keyboard-navigable menus** — arrow keys/Tab, not number entry alone
- **Persistent data in binary files** (`.bin` / `.dat`) — records survive a restart
- **Complexity:** Big-O of every operation, in code comments **and** the report; a measurement table for at least two structures

5.3 Rules for both checks — engineering

- **Tests and docs:** unit-test coverage **100%**, documentation coverage **100%**
- **Platforms:** builds and runs on both **Windows** and **WSL/Linux**
- **GitHub:** private repo, meaningful commits, branches, proper `.gitignore`; **no** compiled files (`.exe`, `.o`, `.class`, `.jar`); Java repo produces a **release**
- **Data:** all data is **synthetic** — no real personal data, ever



6. Deliverables

What you submit, per checkpoint

Deliverable	Midterm	Final
Source code archive (no compiled files)	✓	✓
Report (.docx): design, complexity, measurements, coverage screenshots	✓	✓ (updated)
Doxygen/Javadoc output — PDF only	✓	✓
Test coverage report (HTML, inside the archive)	✓	✓
Presentation (≤ 10 slides)	—	✓
Video (≤ 4 min per member)	—	✓
Live demo and questions (~ 10 min per team)	week 7	week 15

Archive structure — midterm (C)

```
cen207-midterm-name-surname.zip
├── cen207-project-name-surname-c/
│   ├── lib/          # V1-V6
│   ├── app/          # console menus, uses lib
│   ├── test/         # GoogleTest, uses lib
│   ├── CMakeLists.txt
│   ├── report/cen207-midterm-name-surname.docx
│   ├── docs/cen207-midterm-name-surname-doxygen.pdf
│   ├── coverage/     # gcov/lcov HTML
│   ├── .gitignore
│   └── README.md
```

Archive structure — final (Java)

```
cen207-final-name-surname.zip
├── cen207-project-name-surname-java/
│   ├── src/main/java/...    # F1 port of V1-V6, plus F2-F9
│   ├── src/test/java/...    # JUnit 5
│   ├── pom.xml
│   ├── report/, docs/, coverage/
│   ├── presentation/cen207-final-name-surname.pptx
│   ├── video/               # added before zipping, not committed
│   ├── .gitignore
│   └── README.md
```

Naming everything

Name every file with the **course code**, the **checkpoint**, and your **name-surname**:

```
cen207-midterm-name-surname.zip · cen207-final-name-surname.zip ·  
cen207-midterm-name-surname.docx
```

Repository names follow the pattern from the tools/setup section.



7. Team Workflow and Engineering Practices

GitHub Flow

- `main` is protected — all work happens on **feature branches** (`feature/hash-table`, `fix/avl-rotation`)
- Branches merge through **pull requests**, not direct pushes to `main`

Commits and review

Conventional Commits: `feat(lib): implement AVL rotation` ·

`fix(app): correct menu navigation` · `test(hash): add collision unit tests`

Pull requests: open from your branch to `main`; your teammate reviews what changed and how it was tested, then approves before merging

Issues, CI, tags, Definition of Done

- **GitHub Issue** for every requirement code (V1–V6, F1–F9); tracked on a Projects board (Backlog → In progress → In review → Done)
- **CI (if enabled)**: never merge while build/tests are red
- **Version tags**: `midterm-v1.0`, `final-v1.0` for what you submit
- **Definition of Done**: builds without warnings, full test coverage, complexity documented, teammate has reviewed the code



8. Rubrics

Achievement levels (every criterion)

Level	Meaning
5 — Excellent	Everything works, tested, documented; explained step by step in the demo
4 — Good	Minor gaps or edge-case bugs; complete and tested overall
3 — Adequate	Basic operations work; clear gaps in tests/docs/measurements
2 — Poor	Compiles, but most operations wrong or missing; weak explanation
1 — No evidence	Not submitted or not working

Points = (level ÷ 5) × criterion points.

8.1 Midterm rubric — RAP1 (C, 100 points)

#	Criterion	Scope	Points
1	Linear structures	V1 lists, V2 sparse matrix, V3 stack/queue	20
2	Tree and heap	V4 traversals, heap, heap sort	15
3	Graph and traversal	V5 representation, BFS/DFS	15
4	Search and hashing	V6 binary search, hashing, collisions	10
5	Complexity analysis	Big-O + measurement table	10
6	Problem analysis, structure choice	"Why this structure?" in the report	10
7	Software engineering	CMake, tests 100%, docs 100%, GitHub	20

8.2 Final rubric — RAP2 (Java, 100 points)

#	Criterion	Scope	Points
1	Port + integrated app	F1 generics, one menu, binary files	10
2	Graph algorithms	F2, two algorithms	10
3	Sorting	F3, three algorithms, comparison	10
4	Balanced trees	F4 BST/AVL, rotations, range query	15
5	Strings and structures	F5 search/alignment, F6 trie/union-find	15
6	File organisation	F7 files, F8 B+ index, F9 hashing/sort	20
7	Engineering + presentation	Maven, tests 100%, docs, video, demo	20



9. Acceptance Conditions

Submissions are NOT accepted if...

- No GitHub repository, or it is not private, or a team member has no commits
- Unit-test or documentation coverage is **below 100%**
- The repository/archive contains compiled files, or the Java repo has no release
- The application does not build and run on **Windows or WSL/Linux**
- **Plagiarism** is detected

10. Questions Asked in the Demo

Demo questions — Git/GitHub and setup

- Repository created from the template with the correct name? Both members have commits, branches used?
- How were merges and conflicts resolved? Is `.gitignore` correct?
- Build and run on **Windows and WSL**; show `lib` / `app` / `test` and their dependencies

Demo questions — data structures and tests

- Explain a chosen operation (e.g. doubly-linked-list delete, AVL rotation, hash collision) **line by line**, with a box-and-arrow memory drawing
- What is its complexity, and why?
- Open the test/documentation coverage reports; show an edge-case test

Demo questions — file ops and programming

- Add a record, close and reopen the program, show it comes back from the binary file
- **C:** pointers/arrays, `struct`, `malloc` / `free`, file I/O, debugger call stack
- **Java:** generics, interfaces, exceptions

11. Professional Responsibility & Integrity

Ethics, license, synthetic data

- **Ethics:** IEEE/ACM codes ask for honesty about what your code does and does not do, and for giving/accepting fair technical criticism
- **License and attribution:** cite external sources in the report; mark the source/license of any reused code snippet in a comment
- **Synthetic data:** all data is sample data you generate — never real personal data (see rule 5.3)

Plagiarism and shared understanding

- The code and report belong to **your team**
- Copying from another team, a previous term, or an online project is **plagiarism** — similarity checks run, and plagiarism gives **zero points**
- Every member must be able to explain **all** of the team's code — code that cannot be explained in the demo is **not graded**

12. Frequently Asked Questions

FAQ — team and topic

Work alone instead of a team? Yes — at most 5, working alone is allowed. Teams are fixed at the end of week 3.

Two teams want the same topic? First to write it in the Teams table gets it.

Change topic after approval? No — it is fixed together with the project plan.

FAQ — coverage and submission

Coverage below 100%? Not accepted (see acceptance conditions) — reach 100% before the deadline.

Do library collections (e.g. `java.util`) count? No — implement the basic operations yourself.

Compiled files in the repo/archive? No — remove with `.gitignore`; their presence is grounds for rejection.



Appendix — Project Topic List

Eight topic groups

Range	Theme
001–025	Transport, maps and routing
026–050	Games and puzzles
051–075	Text, language and search
076–100	Science, health and bioinformatics
101–125	Networks and computer systems
126–150	Logistics, manufacturing and commerce
151–175	Media, social networks and culture
176–200	City, environment, disasters and agriculture

How a topic box reads

- A short **summary** of the application
- Then, for **every** requirement code (V1–V6 and F1–F9), one sentence: what it stores and what operation it performs — **in that application**
- Same requirement, different story each time

Example — 001 Metro Network Route Planner (1/2)

A console app merging a city's metro/tram/funicular lines into one network; suggests the fastest or fewest-transfer route (~12 lines, 250 stations).

- **V1** Each line: a doubly linked list of stations; ring lines use a circular list
- **V2** Time-slot × station density table stored as a sparse matrix
- **V3** Route adjustments undone with a stack; turnstile entries queued
- **V4** Fault reports kept in a heap by severity
- **V5** BFS finds the fewest-stop route; DFS finds regions cut off by a closure
- **V6** Station name → record in a hash table; binary search over sorted codes

Example — 001 Metro Network Route Planner (2/2)

- **F2** Dijkstra: fastest route in minutes; Kruskal: cheapest new-line network
- **F3** Route options sorted by duration/transfers/distance, three algorithms compared
- **F4** Departure times per station in an AVL tree — fast "next train after X" queries
- **F5** Typed station name matched with KMP; misspelling corrected via edit distance
- **F6** Station names autocomplete via a trie; union-find groups reachable regions
- **F7–F9** Trip records (sequential file), station cards (direct-access + Brent's method), a B+ tree index on card number

Where to find the other 199

The full appendix — all eight groups, all 200 topic boxes — is on the **project guide page**, right after this deck's content.

Browse it, pick one, and write it in the team-and-topic table.

Questions?

Course website: ucoruh.github.io/ce205-data-structures

Next: browse the full topic list on the project guide page